

Ryhmä 05 - A.L.B.E.R.T.

Sisällysluettelo:

- 1. Ryhmä
- 2. Johdanto
 - 2.1. Idea
 - 2.2. Tavoitteet
- 3. Tekninen toteutus
 - 3.1. Tuulivoimala
 - 3.1.1. Kuvaus
 - 3.1.2. Haasteet
 - 3.1.3. Komponentit
 - 3.1.4. 3D-tulostetut osat
 - 3.1.5. Tuulivoimalan ja tasasuuntaajan kytkentäkaaviot
 - 3.2. Tuulensuuntamittari
 - 3.2.1. Kuvaus
 - 3.2.2. Haasteet
 - 3.2.3. Komponentit
 - 3.2.4. 3D-tulostetut osat
 - 3.3. Valvontajärjestelmä
 - 3.3.1. Kuvaus
 - 3.3.2. Haasteet
 - 3.3.3. Komponentit
 - 3.3.4. 3D-tulostetut osat
 - 3.3.5. KytKentäkaavio
 - 3.4. Tuulivoimalan rasia
 - 3.5. Kehityskuvat
- 4. Lopputulos
 - 4.1. Kuvat
 - 4.2. Esittelyvideo
- 5. Kommentoitu ohjelmakoodi
- 6. API:n linkit
- 7. Kehitysideat
- 8. Lähteet

1. Ryhmä

Jäsenet/Members:

nimi	pääaine
Jussi Nurmi	Sähköenergiatekniikka
Eino Hannunen	Elektroniikka ja sähköfysiikka
Tsvetomir Zhivkov	Tietotekniikka
Alexey Volkov	Sähköenergiatekniikka

2. Johdanto

2.1. Idea

"Aerodynamical Logical Brutal Electrical Rotating Turbine" (= A.L.B.E.R.T)

Ryhmämme rakentui energiatekniikasta kiinnostuneiden opiskelijoiden ympärille. Perusideaksi valikoitui tuulivoimalan toiminnan mallintaminen pienessä mittakaavassa. Tämän kautta pyrimme sisällyttämään projektiin mahdollisimman laaja-alaista oppimista pienelektroniikasta ja ohjelmoinnista aina voimalatekniikan ja tehoelektroniikan perusperiaatteisiin asti.

Ideasta teki houkuttelevan myös tuulivoimalan ja siihen mahdollisesti liittyvän energianvarastointikapasiteetin, valvontajärjestelmän sekä sähköverkon selkeän modulaarinen rakenne: Osakokonaisuuksia oli mahdollista lisätä tai pudottaa pois kurssin edetessä, jolloin tekeminen skaalautui luonnollisella tavalla käytössä olevaan aikaresurssiin. Saatoimme olla jo projektin alussa melko varmoja, että pääsemme loppunäytöksessä esittämään jonkinlaisen toimivan kokonaisuuden.

2.2. Tavoitteet

- Rakentaa pienoiskoossa tuulivoimala, joka sisältää kaikki oleelliset peruskomponentit (torni, generaattori ja roottori).
- Saada roottori suuntautumaan tuulen suunnan mukaan (muuntaen mahdollisimman suuren osan virtauksen liike-energiasta sähköksi).
- Suunnitella valvontajärjestelmä, joka tarkkailee tuulivoimalan tilaa.
- Muuntaa generaattorin jännite käyttökelpoiseen muotoon.
- Varastoida saatu sähkö akustoon (**ei toteutunut**)

3. Tekninen toteutus

3.1. Tuulivoimala

3.1.1. Kuvaus

Tuulivoimalan ulkokuori koostuu 3D-tulostimella valmistetusta tornista ja nasellista. Naselli (eli voimalan konehuone) pitää sisällään kolmivaihegeneraattorin, vaihteiston sekä pyörimisakselin, joka siirtää roottorin lapojen talteen ottaman tuulen liike-energian pyörimisenergiana generaattoriin. Naselli on kytketty askelmoottoriin, joka pystyy kääntämään konehuonetta vaakatasossa noin 240 asteen alueella käyttäen virtalähteenään kuutta sarjaan kytkettyä 1,2 voltin ladattavaa paristoa.

Voimalan säätölaitteiston sydän on ESP32-mikrokontrolleri, jolle tarjotaan DC/DC-muuntimen kautta 3.3 V:n käyttöjännite. Mikrokontrolleri välittää verkossa olevalle palvelimelle generaattorin tuottamaa kokoaaltotasasuunnattua jännitettä sekä askelmoottorin paristojen tilaa seuraavien jänniteantureiden tiedot. Kontrolleri toimii myös linkkinä tuulensuuntamittarin ja askelmoottoria ohjaavan TMC2209-piirin välillä. Askelmoottorin ohjaus perustuu tuulensuuntamittarin Hall-anturin tuottamaan kulmadataan.

3.1.2. Haasteet

Generaattorin johtimet:

Koska halusimme, että tuulivoimalan naselli kääntyy, meidän piti huomioida generaattorin johtojen mahdollinen sotkeentuminen. Ratkaisuna haasteeseen muokkasimme tornin sisäosia pyöreämmäksi sekä rajoitimme pyörimissuuntaa kahdella mikrokytkimellä. Mikrokytkimien avulla pystyimme myös kalibroimaan turbiinin suunnan sekä käynnistyksessä että tilanteissa, joissa stepperin toiminta häiriintyy.

3-vaihegeneraattorin tasasuuntaaja sekä jänniteanturit

Koska alkuperäisessä suunnitelmassa generaattorin jännitettä oli tarkoitus hyödyntää, tarvitsimme 3-vaihegeneraattoriin AC/DC-tasasuuntaajan. Peruseriaate löytyi internetistä varsin helposti (full-wave rectifier) ja rakensimme sen pohjalta kuuteen Schottky-diodiin perustuvan kytkennän. Suurten jännitevaihteluiden vaimentamiseksi piiriin piti vielä lisätä kuorman rinnalle sopiva kondensaattori.

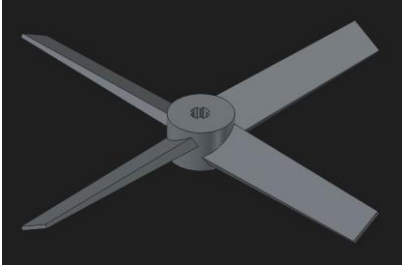
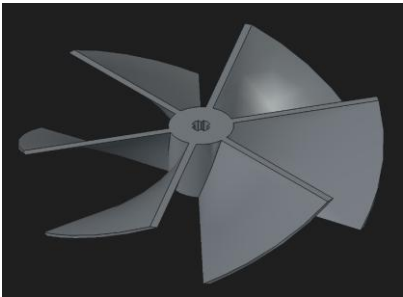
Tarvitsimme myös jänniteantureita generaattorin tuottaman jännitteen ja paristojen jännitteen mittaamiseen. Koska sähköpajalta ei löytynyt valmiita komponentteja, päätimme rakentaa anturit itse.

Jänniteanturi toteutettiin yksinkertaisella kahteen vastuksen pohjautuvalla jännitejakajalla. Mittasimme voimalan generaattorin maksimijännitteen ja laskimme sen perusteella sopivat vastusarvot niin, että jännite saatiin skaalattua ESP32:n 3,3 voltin maksimisisääntuloon. Lisäksi

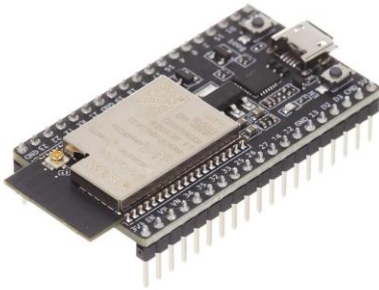
tarkistimme, että vastusten tehohäviö ei ylitä 0.25 wattia. Huomioimme myös piirin lähtöimpedanssin, sillä sen tuli olla alle 10 kilo-ohmia ESP32:n dokumentaation mukaisesti.

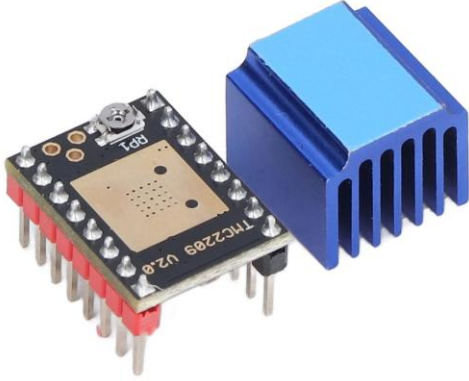
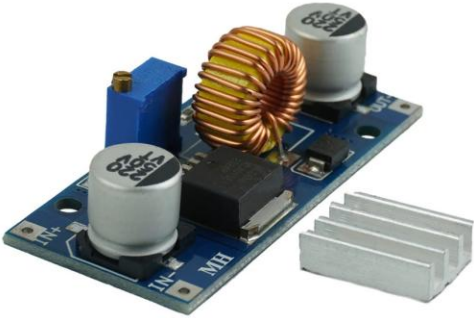
Roottori

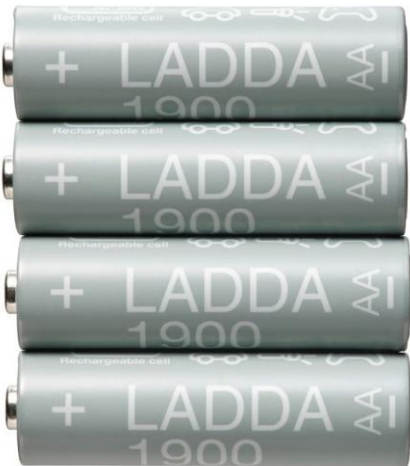
Ensimmäinen kehitysversiomme ei tuottanut tarpeeksi vääntöä pyörittääkseen roottoria vaihdelaatikon välityksellä. Vaihtoehtoisia siipiprofiileja oli tarjolla useita, mutta päädyimme toteutuksessa tietokoneen tuuletinta muistuttavaan muotoon, joka reagoi erityisen suotuisasti testauksessa käytettyyn kapeaan virtaussuihkuun. Uusi roottoriprofiili kasvatti generaattorista saadun tehon lähes kymmenkertaiseksi.

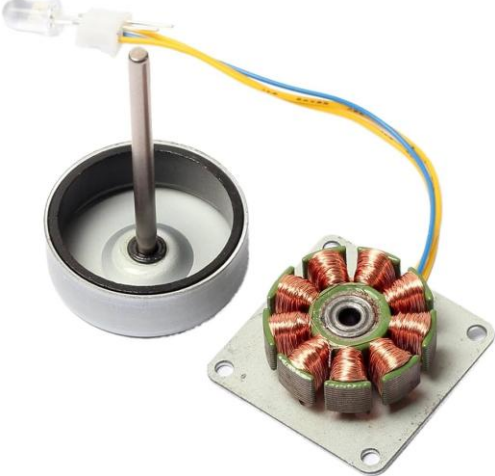
Vanha roottori	Uusi roottori
	

3.1.3. Komponentit

Komponentti	Määrä	Kuva
ESP32 WROVER-IE	1	

TMC2209 Stepperi ohjain	1	
Askelmoottori	1	
Step Down Converter (DC/DC- muunnin)	1	

USB-liitin (naaras)	1	
Paristokotelo (6x AA)	1	
Ladattavat paristot	6	

3- vaihegeneraattori	1	 A 3-phase generator assembly consisting of a white cylindrical housing with a central shaft, and a separate stator with copper windings on a metal base. Two yellow and blue wires are connected to the stator.
Mikrokytkin	2	 A black microswitch with a metal lever on top and two metal terminals at the bottom.
Kytkin	1	 A blue push-button switch with a metal shaft and three metal terminals at the bottom.

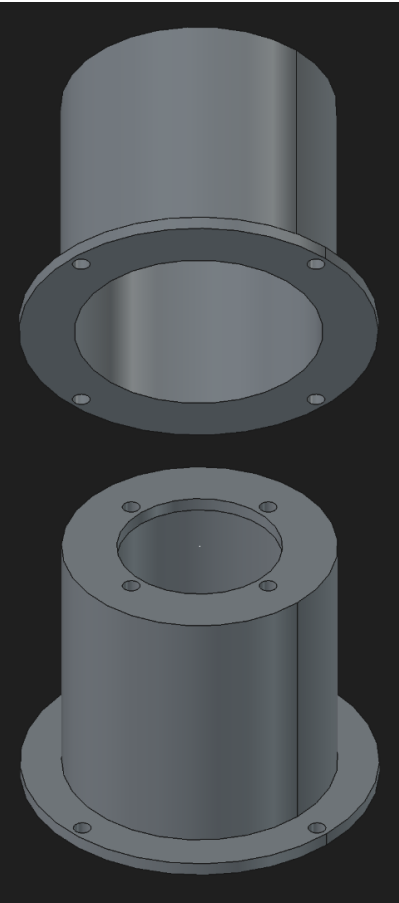
100 nF kondensaattori	2	
3300 uF kondensaattori	1	
Schottky-diodi 1N5817	6	

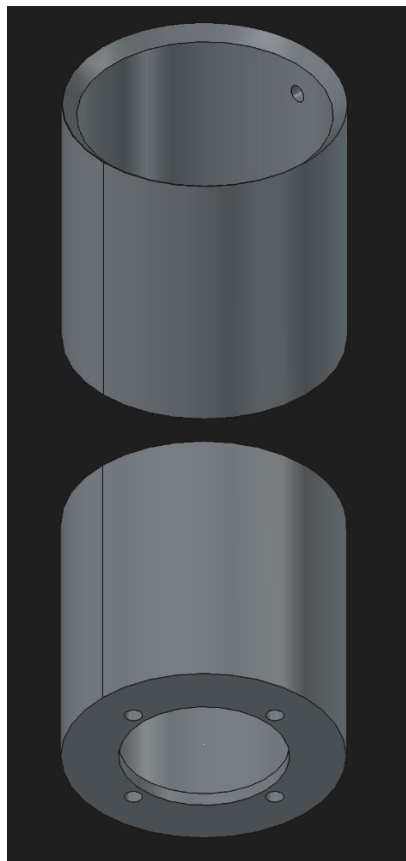
Vastukset: • 2x 1.0 kΩ • 1x 2.7 kΩ • 1x 1.8 kΩ	4	
4-napainen riviliitin	2	
Johtimet	n. 70	

12x8x3 Laakerit	4	
M6 pultit	10	
M6 mutterit	10	

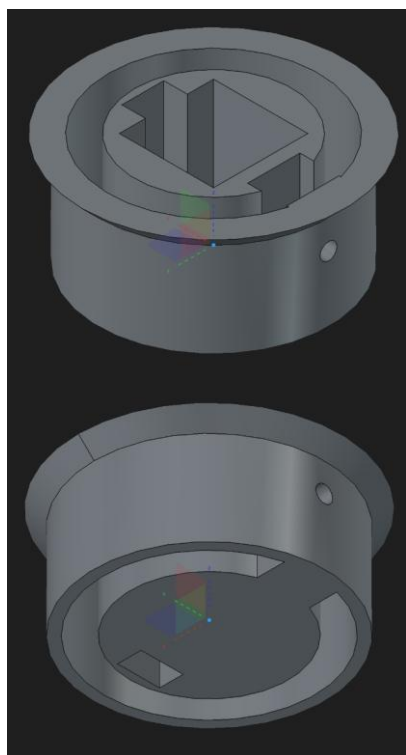
M3 pultit	4	
M3 mutterit	4	
USB A - micro USB kaapeli	1	

3.1.4. 3D-tulostetut osat

Kuva	Kuvaus
	Tuulivoimalan tornin pohja. Kiinnitetään tuulivoimalan rasiaan ja tornin huippuun M6 pulteilla



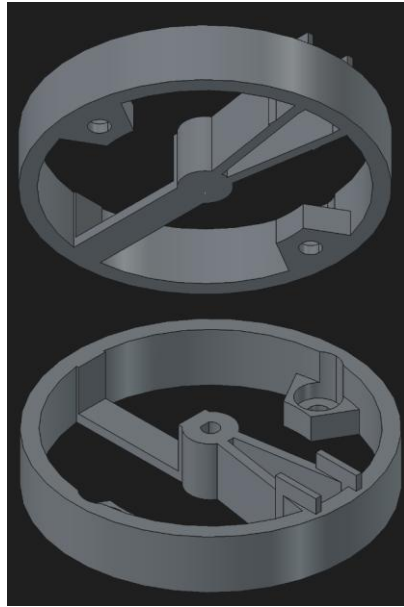
Tuulivoimalan tornin huippu.



Askelmoottorin asema.

Asetetaan tornin huipun sisään.
Kiinnitys torniin yhdellä M6
pultilla.

Askelmoottori sopii aseman
keskellä olevaan neliön
muotoiseen reikään.

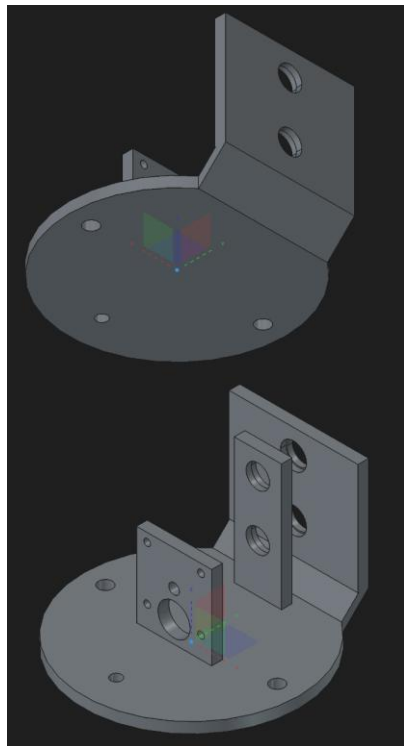


Nasellin pohja.

Askelmoottorin varsi kiinnittyy pohjan keskellä olevaan reikään, ja näin kääntää koko konehuonetta.

Pohjan korotetut osat törmäävät mikrokytkimiin myllyn pyöriessä.

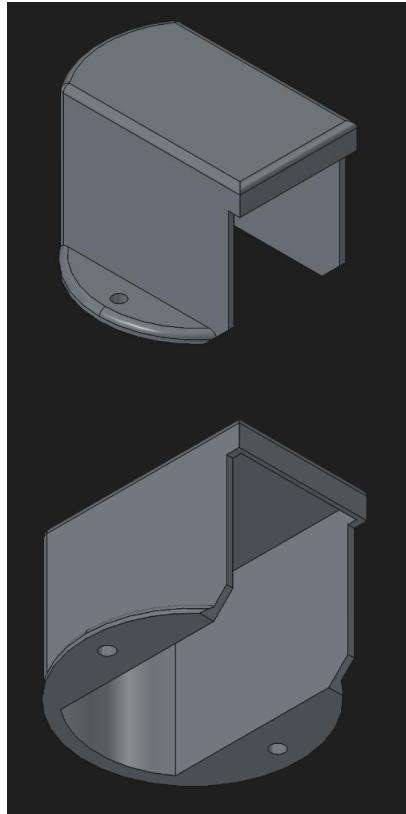
Generaattorin johdot pääsevät laskeutumaan korotettujen osien välistä rungon läpi voimalan rasiaan.



Nasellin sisältämä 1:2 vaihdelaatikko.

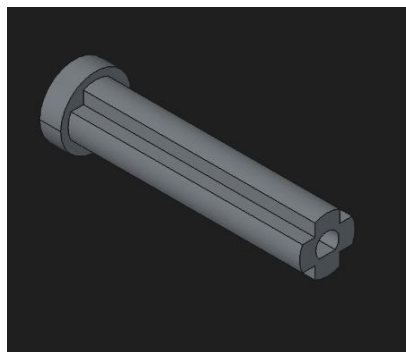
Vaihdelaatikkoon kiinnitetään generaattori, generaattorin vaihdelaatikon akselit, 8- ja 16- hampaiset hammasrattaat sekä neljä laakeria.

Generaattori kiinnittyy alempaan akseliin ja roottori kiinnittyy ylempään akseliin.

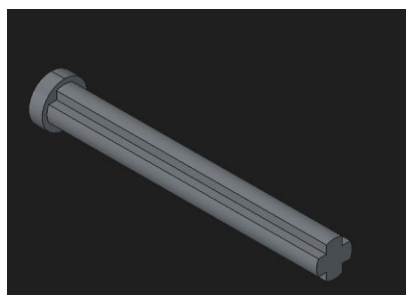


Nasellin kansi.

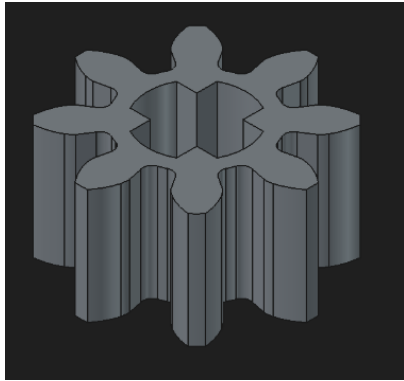
Kansi, vaihdelaatikko ja pohja kiinnittyvät toisiinsa kahdella M6 pultilla. Jokaisesta kappaleesta löytyy samasta kohdasta pulteille reiät.



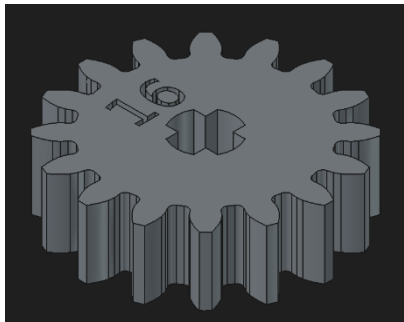
Vaihdelaatikon alempi akseli



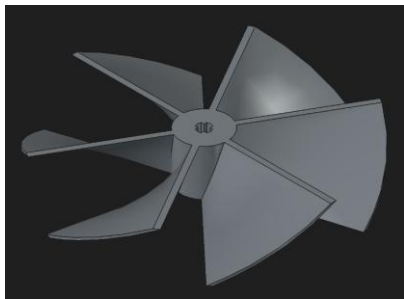
Vaihdelaatikon ylempi akseli



Vaihdelaatikon alempi
hammasratas, 8 hammasta

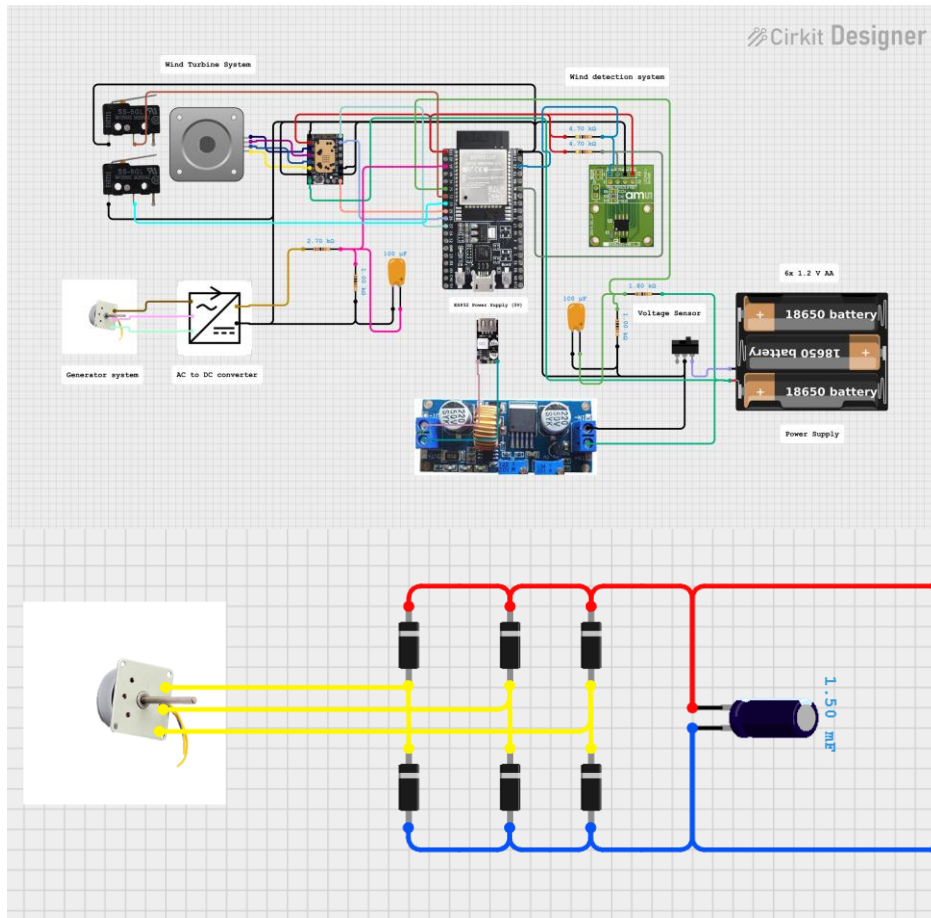


Vaihdelaatikon ylempi
hammasratas, 16 hammasta



Roottori

3.1.5. Tuulivoimalan ja tasasuuntaajan kytkentäkaaviot



3.2. Tuulensuuntamittari

3.2.1. Kuvaus

Tuulensuuntamittari koostuu magneettisesta enkooderista, magneetista, laakerista ja neljästä 3D tulostetusta osasta: siivestä, siiven akselistä, rasian kannesta ja rasian pohjasta. Laakerissa istuvan siiven akselin pohjaan on teipattu magneetti, jonka liikkeen pohjalta enkooderi määrittää tuulen suunnan. Suuntatieto välittyy voimalan toimintaa säätelevälle ESP32-mikrokontrollerille langallisesti.

3.2.2. Haasteet

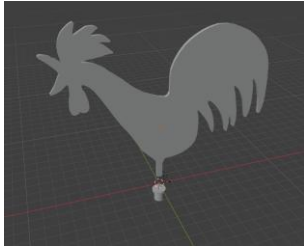
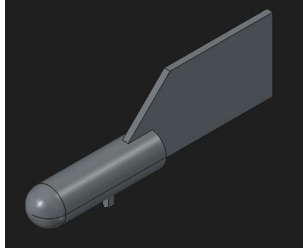
Akseli

Tuulensuuntamittarin akselin ja laakerin välissä tulee olla tiukka sovite, jotta magneetti pysyy kohtisuorassa suhteessa enkooderiin. Mikäli magneetti kallistuu, ei mitattu suunta enää ole tarkka. ADDLAB'in Ultimakerien toleranssit olivat liian suuret tiukan sovituksen vaatimiin.

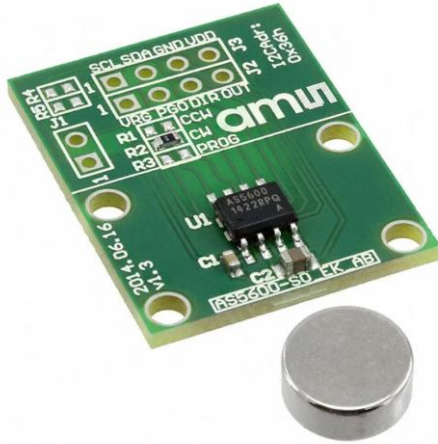
toleransseihin, joten oikean kokoisen akselin saamiseksi jouduttiin tulostamaan huomattava määrä prototyyppejä.

Siipi

Alun perin suuntamittariin piti tulla kukkoa muistuttava siipi. Valitettavasti muoto ei soveltunut tuulen suunnan mittaamiseen; kappale lopulta hajosi testeissä. Kukon tilalle toteutimme tukevamman ja virtaviivaisemman siiven.

Vanha siipi	Uusi siipi
	

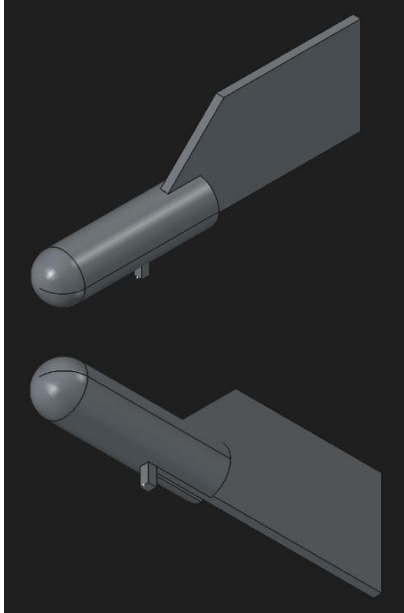
3.2.3. Komponentit

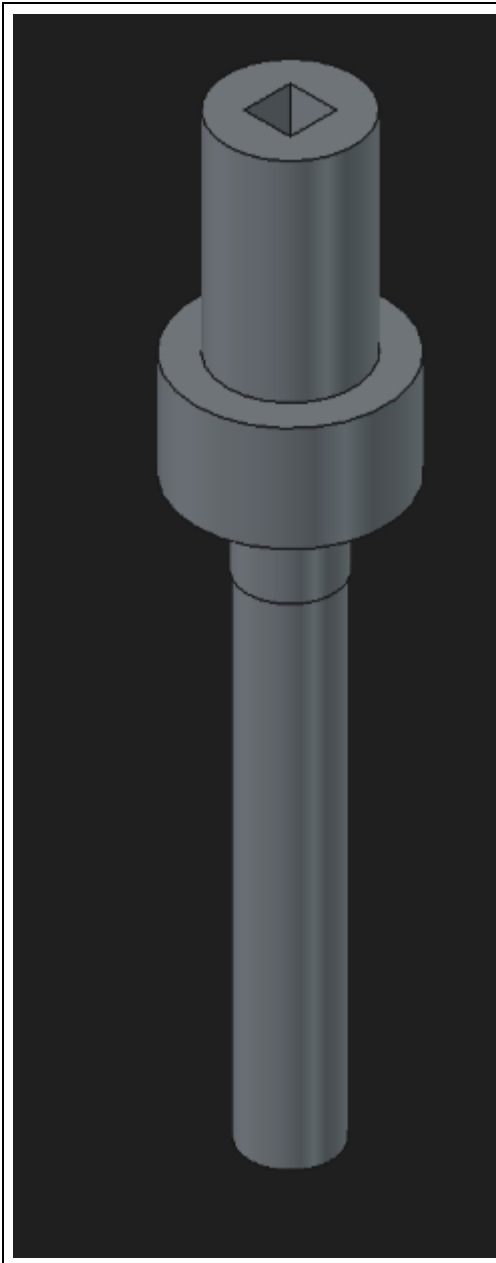
Komponentti	Määrä	Kuva
AS5600 magneettinen enkooderi	1	

Vastukset: ▪ 2x 4.7 kΩ	2	
608 2RS Laakeri	1	
M6 pultit	2	

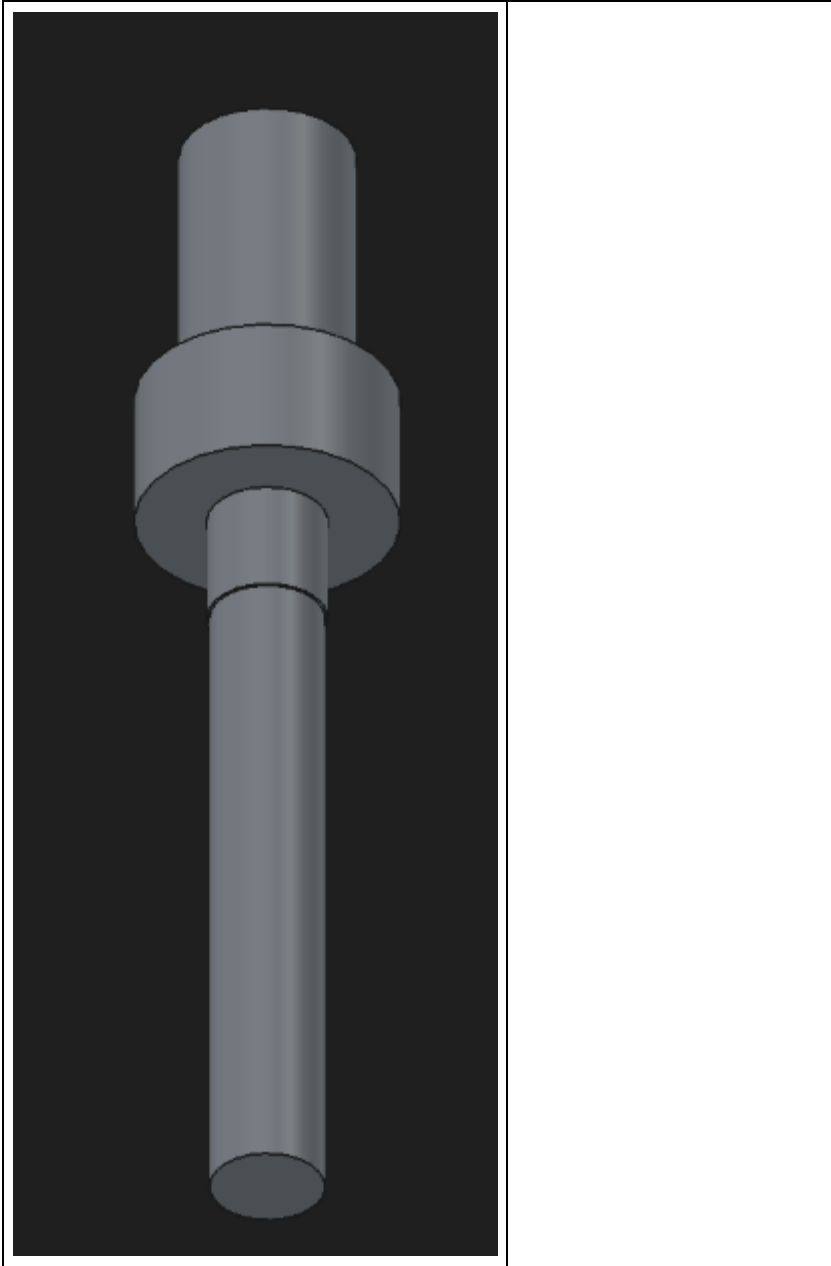
M6 mutterit	2	
-------------	---	---

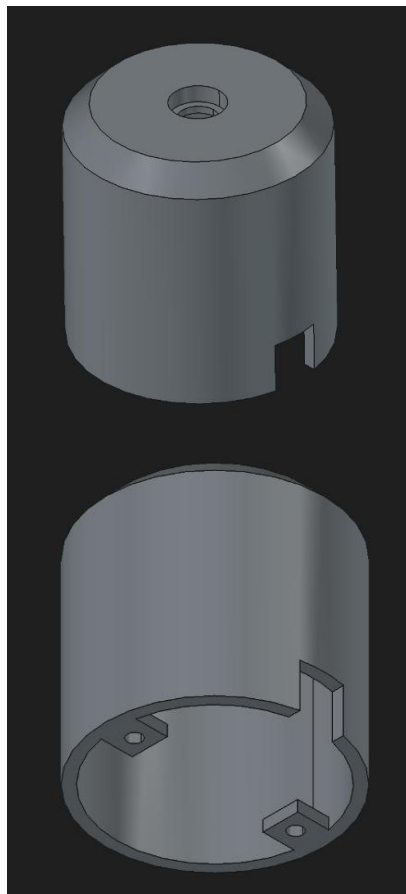
3.2.4. 3D-tulostetut osat

Kuva	Kuvaus
	Tuulensuuntamittarin siipi.

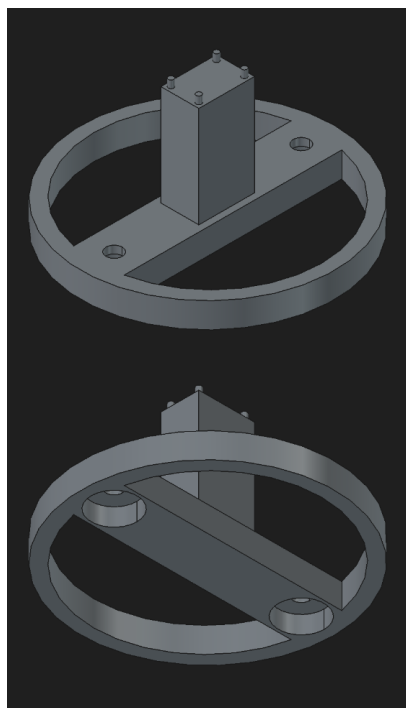


Tuulensuuntamittarin
siiven akseli. Alin
paksunnus kiinnittyy
608 2RS laakeriin
tiukalla
sovitteella. Siipi
lasketaan neliön
muotoiseen reikään
akselin päällä. Akselin
pohjaan teipataan
magneetti, jotta
magnetic encoder
pystyy lukemaan siiven
kulman.





Tuulensuuntamittarin rasian kansi. Kannen päällä olevaan reikään laitetaan 608 2RS laakeri, johon kiinnitetään akseli.



Tuulensuuntamittarin rasian pohja. Pohjan kiinnitys rasiaan toteutetaan M6 pulteilla. Magneettinen enkooderi kiinnitetään pohjassa olevalle korokkeelle teipillä. Korokkeesta nousevat pienet tolpat auttavat enkooderin pitämisessä paikallaan.

3.3. Valvontajärjestelmä

3.3.1. Kuvaus

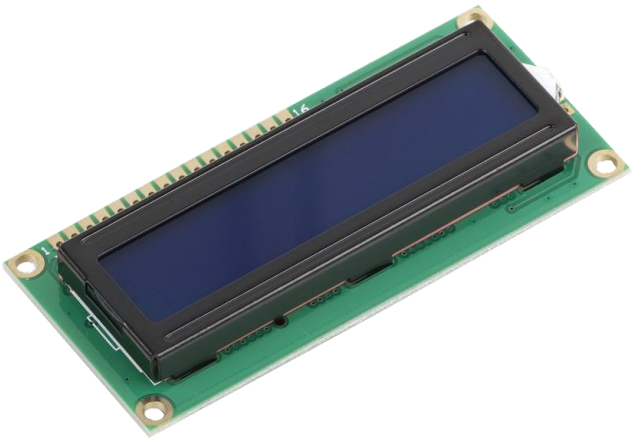
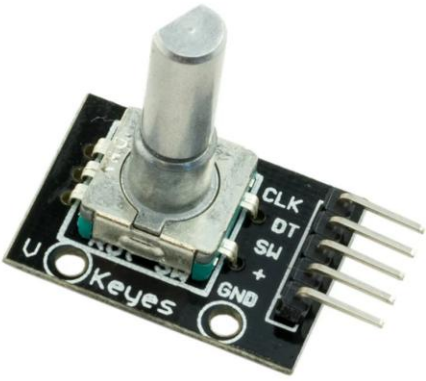
Valvontajärjestelmä koostuu potentiometrasta, LCD-näytöstä, kiertoanturista, LED:istä, ESP32-mikrokontrollerista sekä kotelosta. Järjestelmä noutaa langattomasti Wi-Fi:n kautta verkossa olevalta palvelimelta sensoridataa, jonka se esittää LCD-näytölle tulostuvan valikkorakenteen avulla (perustiedot, tuulen suunta, generoitu jännite, stepperin akun varaus). Järjestelmä sisältää myös näytön kontrastisäädön sekä voimalan jännitteen perusteella aktivoituvan RGB-ledin ($V = 0 \rightarrow$ punainen, $V \neq 0 \rightarrow$ vihreä). Virtalähteenä käytetään USB:n välityksellä esim. Power Bank-tyypistä varavirtalähdettä.

3.3.2. Haasteet

Suurin haaste valvontajärjestelmän toteutuksessa oli ohjelmallinen: sensoridatan nouto verkosta ja siihen käytettävät http-kutsut aiheuttivat huomattavaa viivettä graafisen käyttöliittymän toimintaan, osin jopa jumittivat sen kokonaan. Ratkaisu löytyi EPS32:een integroidun FreeRTOS-käyttöjärjestelmän toiminnallisuuksista, jotka mahdollistivat verkkoon liittyvien toimintojen erottamisen omaksi kokonaisuudekseen.

3.3.3. Komponentit

Komponentti	Määrä	Kuva
ESP32 XXSR69	1	

LCD näyttö	1	
Potentiometri	1	
Kiertoanturi	1	

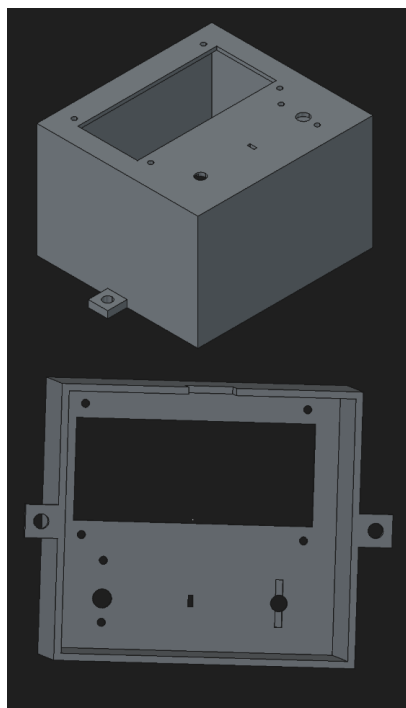
RGB LED	1	
Vastukset: • 3x 220 Ω	3	
Johtimet	n. 20	

M6 pultit	2	
M6 mutterit	2	
M3 pultit	5	

M3 mutterit	5	
Virtapankki	1	
USB A - USB C kaapeli	1	

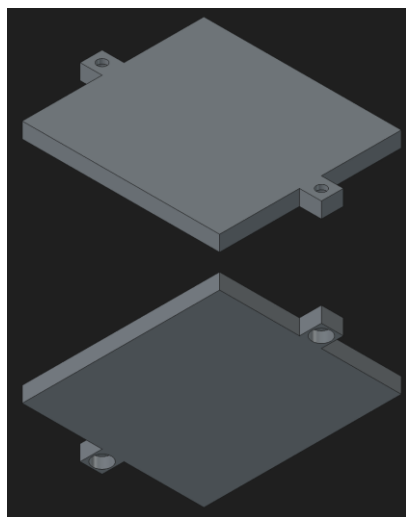
3.3.4. 3D-tulostetut osat

Kuva	Kuvaus
------	--------



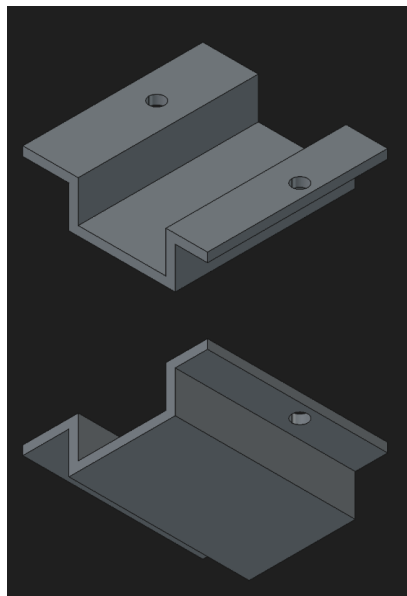
Valvontajärjestelmän rasian kansi.

Kanteen kiinnitetään LCD-näyttö, kiertoanturi, potentiometri ja ledi.

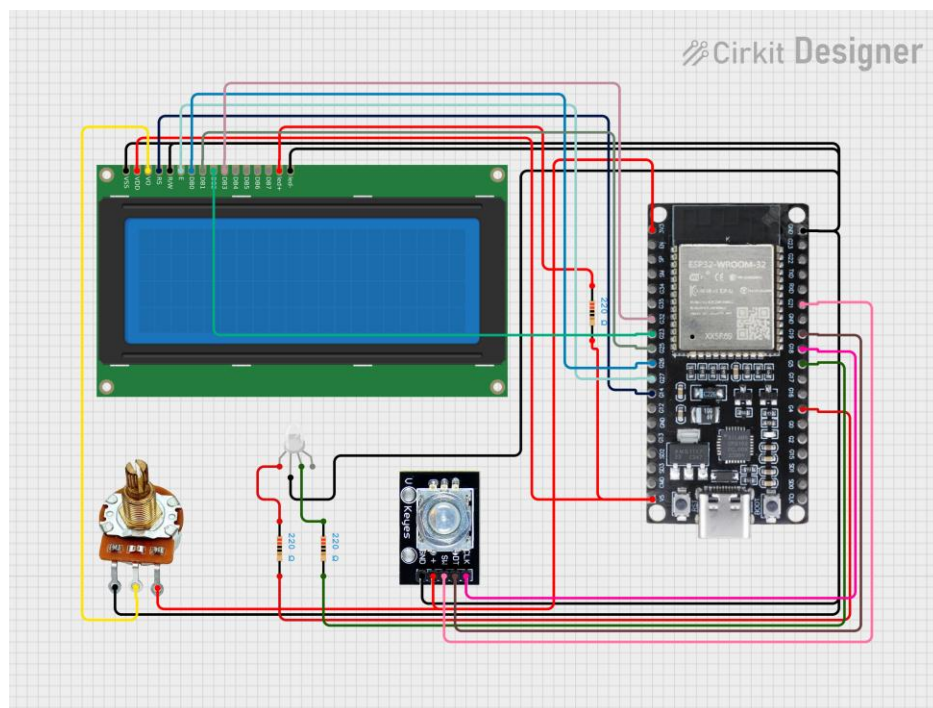


Valvontajärjestelmän rasian pohja.

Kiinnitetään rasian kanteen M6 pulteilla.

	Kiertoanturi tuki. Toinen valvontajärjestelmän kannessa olevista kiertoanturin tuen reistä on väärässä paikassa, joten tuki on kiinni kannessa vain yhdellä ruuvilla. Virheen korjaamiseksi tuki mitoitettiin niin isoksi, että rasian seinät estävät tuen pyörimisen.
---	--

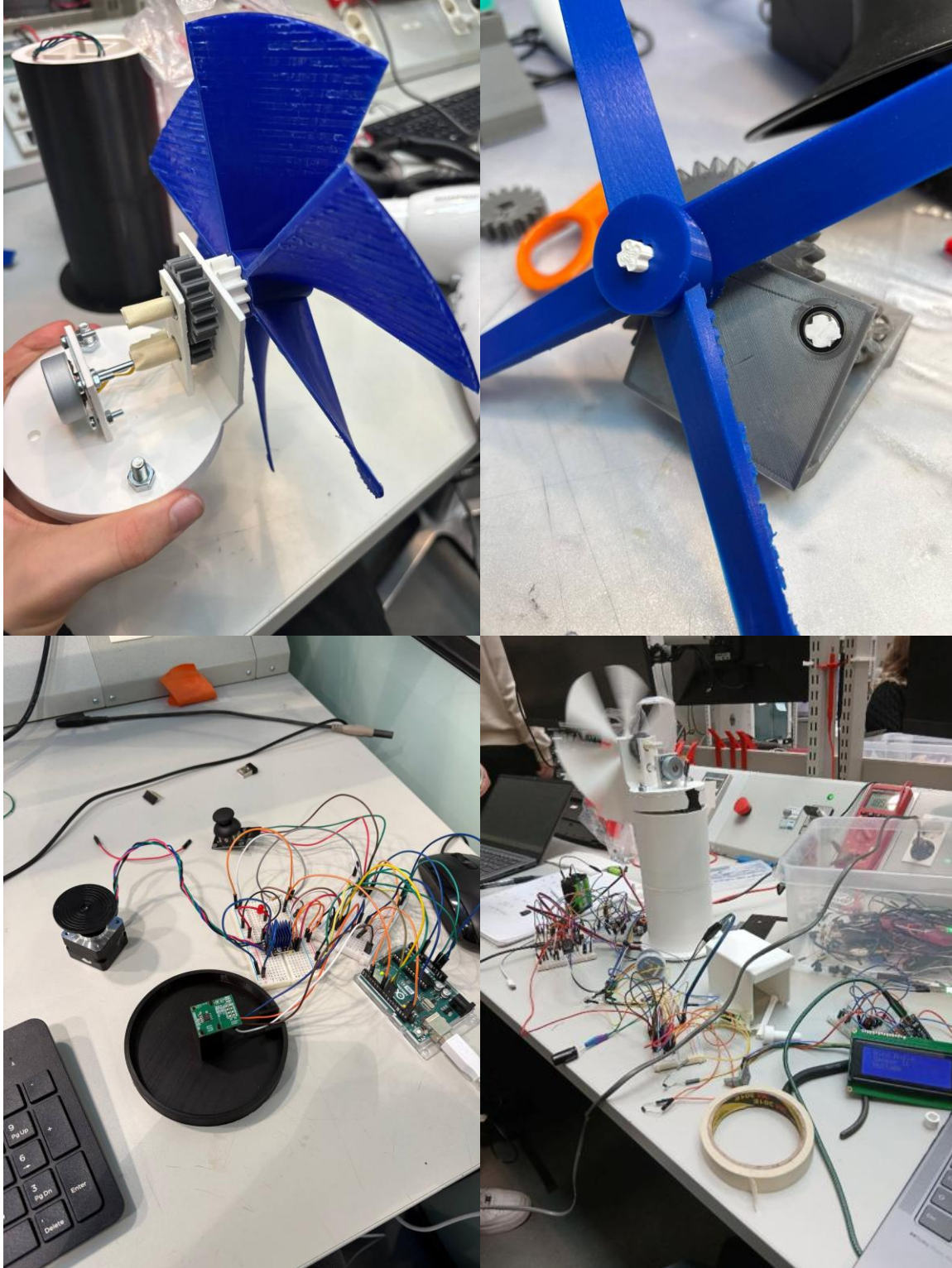
3.3.5. Kytchentäkaavio



3.4. Tuulivoimalan rasia

Tuulivoimalan ja suuntamittarin elektroniset komponentit ovat puisessa rasiassa, jonka päälle on asennettu itse voimala. Tuulivoimalan johdot kytketään rasiaan päällä olevan reiän kautta ja suuntamittarin johdot kyljessä olevan reiän kautta.

3.5. Kehityskuvat



4. Lopputullos

4.1. Kuvat



4.2. Esittelyvideo

<https://youtu.be/IGIYgQ5NQMg>

5. Kommentoitu ohjelmakoodi

Tuulivoimalan koodi + API

GitHub: <https://github.com/tsvetomir-zhivkov/A.L.B.E.R.T..git>

Valvontajärjestelmän koodi

Valvontajärjestelmän koodi

```
#include <Arduino.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <LiquidCrystal.h>

LiquidCrystal lcd(14, 27, 26, 25, 33, 32); //RS=14, Enable=27, D4=26, D5=25,
D6=33, D7=32; LCD-näytön PIN-konfiguraatio

const int RED_PIN = 4;          //LED-valon PIN-konfiguraatio
const int GREEN_PIN = 5;

const int CLK_PIN = 18;         //Pulssianturin PIN-konfiguraatio: liike=18,
suunta=19, kytkin=21
const int DT_PIN = 19;
const int SW_PIN = 21;

int lastCLK = 0; //Liikemuuttujan alustus

int menuIndex = 0; //>"-valitsimen paikkaa seuraava muuttuja
int selectedOption = 0; //Kytkinpainikkeella valittu vaihtoehto
bool inMenu = true; //Muuttuja, joka kertoo, ollaanko päävalikossa

unsigned long lastPress = 0; //Kytkinpainikkeen ajastin / Tietotyyppi valittu
millis()-funktion perusteella
const unsigned long buttonDebounce = 200; //Kahden painalluksen minimiväli
200 ms; vakauttaa kytkimen toimintaa

unsigned long lastUpdate = 0; //Näytön päivityksen ajastin
const unsigned long updateInterval = 1500; // Mittausdatan piirtotaajuus;
estää ruudun häiritsevän tiheän vilkunnan

const char* ssid = "A16"; //Wifi-komentojen edellyttämä tietotyyppi 'char*'
const char* password = "*****";

String turbineInfo[3]; //Turbiinin perustiedot
float voltage;
int windReading;
```

```

int battery;

//TÄRKEITÄ FUNKTIOITA//

void sensorTask(void *pvParameters) {      //Hyödynnetään ESP32:n toista
ydintä / luodaan sensoridatan lukemista varten oma 'tehtävä'
    while(true) {
        voltage = getMeasurement(3).toFloat(); //getMeasurement tekee http-
kutsun saadakseen generaattorin jännitteen; nyt se ei sotke itse
käyttöliittymää
        windReading = getMeasurement(1).toInt();
        battery = getMeasurement(4).toInt();

        vTaskDelay(1500 / portTICK_PERIOD_MS); //Rajoittaa http-kutsujen määrää
    }
}

String getTurbineData() { //Hakee turbiinin perustiedot; ajetaan pelkästään
järjestelmän käynnistyksessä

    HTTPClient http; //Alustetaan HTTP-olio, jolla on käytössään
http-yhteyden kannalta oleellisia funktioita
    http.begin("https://albert2026.azurewebsites.net/api/wind_turbine
s/4"); //Alustaa yhteyden

    int httpCode = http.GET(); //Pyytää palvelimelta dataa

    if (httpCode == HTTP_CODE_OK) {

        String payload = http.getString(); //Tallentaa palvelimelta
ladatun merkkijonon paikalliseen muuttujaan

        http.end(); //Sulkee yhteyden palvelimeen

        //DATALEIKKURI//

        String key = "\"name\": \""; //Alustaa avaimen, jolla
leikkauskohta löydetään
        int start = payload.indexOf(key); //Alustaa muuttujan, joka
saa arvokseen leikkauskohdan aloitusindeksiin

        if (start != -1) { //indexOf-funktio palauttaa arvon -1, jos
avainta ei löydy
            start += key.length();
            int end = payload.indexOf("\"", start); //Lopettaa
näytteenoton heittomerkkiin
            if (end != -1)
                turbineInfo[0] = payload.substring(start, end);
//Lisää leikatun datan globaaliin listaan
        }

        key = "\"height\": \""; //Turbiinin korkeus
        start = payload.indexOf(key);

        if (start != -1) {
            start += key.length();

```

```

        int end = payload.indexOf("\"\"", start);
        if (end != -1)
            turbineInfo[1] = payload.substring(start, end);
    }

    key = "\"rotor_diameter\": \""; //Roottorin halkaisija
    start = payload.indexOf(key);

    if (start != -1) {
        start += key.length();
        int end = payload.indexOf("\"\"", start);
        if (end != -1)
            turbineInfo[2] = payload.substring(start, end);
    }

    }

    http.end();
    return ""; //Varmistaa, että virhetilanteessakin funktiolla on
palautusarvo
}

String getMeasurement(int sensorID) { //Mittauksien nouto palvelimelta;
jokaisella sensorilla on oma hakemisto

    HTTPClient http;
    http.begin("https://albert2026.azurewebsites.net/api/wind_turbine
s/4/sensors/" + String(sensorID) + "/logs" );
    http.setTimeout(500); //Rajoittaa sitä, kauanko järjestelmä
yrittää saada palvelimeen yhteyden

    int httpCode = http.GET();

    if (httpCode == HTTP_CODE_OK) {

        String payload = http.getString();

        http.end();

        //DATALEIKKURI//

        String measurement = "";
        String key = "\"measurement\": \""; //Ks. getTurbineData-
funktion kommentit; identtinen toteutus
        int start = payload.indexOf(key);

        if (start != -1) {
            start += key.length();
            int end = payload.indexOf("\"\"", start);
            if (end != -1)
                measurement = payload.substring(start, end);
        }

        return measurement;
    }

    http.end();

```

```

        return "";
    }

String windDirection() { //Konverttoi tuulen suunnan asteissa ilmansuunnaksi
    if (windReading >= 338 || windReading < 23) {
        return "N";
    }
    else if (windReading >= 23 && windReading < 68 ) {
        return "NE";
    }
    else if (windReading >= 68 && windReading < 113 ) {
        return "E";
    }
    else if (windReading >= 113 && windReading < 158 ) {
        return "SE";
    }
    else if (windReading >= 158 && windReading < 203 ) {
        return "S";
    }
    else if (windReading >= 203 && windReading < 248 ) {
        return "SW";
    }
    else if (windReading >= 248 && windReading < 293 ) {
        return "W";
    }
    else if (windReading >= 293 && windReading < 338 ) {
        return "NW";
    }
    return "";
}

```

```

void drawMenu() { //Valikon piirtoasusta vastaava funktio

    lcd.clear(); //Tyhjentää ruudun

    lcd.setCursor(0,0); //Asettaa piirturin 1. riville
    if (menuIndex == 0) { //Piirtää osoittimen ">" menuIndex-muuttujan
    osoittamalle riville
        lcd.print("> ");
    }
    else {
        lcd.print(" ");
    }
    lcd.print("Turbine");

    lcd.setCursor(0,1); //Asettaa piirturin 2. riville
    if (menuIndex == 1) {
        lcd.print("> ");
    }
    else {
        lcd.print(" ");
    }
    lcd.print("Wind Direction");

    lcd.setCursor(0,2);
    if (menuIndex == 2) {

```

```

        lcd.print("> ");
    }
    else {
        lcd.print(" ");
    }
    lcd.print("Generator");

    lcd.setCursor(0,3);
    if (menuIndex == 3) {
        lcd.print("> ");
    }
    else {
        lcd.print(" ");
    }
    lcd.print("Battery");
}

void drawOption() { //Kytkeinpainikkeelle valitun vaihtoehdon piirtäminen

    lcd.clear();

    if (selectedOption == 0) { //Tarkistaa selectedOption-muuttujan tilan ja
piirtää sen mukaan halutun mittausdatan
        lcd.setCursor(0,0);
        lcd.print("Turbine: " + turbineInfo[0]);
        lcd.setCursor(0,1);
        lcd.print("Height: " + turbineInfo[1] + " cm");
        lcd.setCursor(0,2);
        lcd.print("Rotor: " + turbineInfo[2] + " cm");

    }

    else if (selectedOption == 1) {

        lcd.setCursor(0,0);
        lcd.print("Wind Direction:");
        lcd.setCursor(0,1);
        lcd.print(windDirection());
    }

    else if (selectedOption == 2) {

        lcd.setCursor(0,0);
        lcd.print("Generator(V):");
        lcd.setCursor(0,1);
        lcd.print(voltage);
    }

    else if (selectedOption == 3) {

        lcd.setCursor(0,0);
        lcd.print("Battery(%):");
        lcd.setCursor(0,1);
        lcd.print(battery);
    }
}
}

```

```

void setup() {

    lcd.begin(20, 4);    //Alustaa LCD-näytön / 20 saraketta ja 4 riviä

    WiFi.begin(ssid, password); //Yhdistää annettuun Wifiin

    while (WiFi.status() != WL_CONNECTED) { //Odottaa, että yhteys muodostuu;
turbiinin perustietojen nouto vaatii yhteyden
        delay(500);
    }

    pinMode(RED_PIN, OUTPUT); //Määrittää punaisen ledin jännitteen
ulostuloksi
    pinMode(GREEN_PIN, OUTPUT);
    pinMode(CLK_PIN, INPUT_PULLUP); //Määrittää pulssianturin signaalin
lähteeksi ja kytkee siihen sisäisen ylösvetovastuksen
    pinMode(DT_PIN, INPUT_PULLUP);
    pinMode(SW_PIN, INPUT_PULLUP);

    Serial.begin(115200); //Alustaa kommunikaation koneen ja ESP:n välille

    lastCLK = digitalRead(CLK_PIN); //Lukee pulssianturin alkutilan

    drawMenu(); //Piirtää LCD-näytölle alkuvalikon
    getTurbineData(); //Hakee tuuliturbiinin perustiedot

    xTaskCreatePinnedToCore( /*Luo 'tehtävä'-silmukan, joka toimii
riippumattomasti muun toiminnallisuuden taustalla. Tämän käyttö oli tekoälyn
ehdotus;

                                koin sen konseptina äärimmäisen hyödylliseksi
ja painoin sen mieleeni helpottamaan vastaavien projektien toteutusta
tulevaisuudessa */
        sensorTask,          // 'tehtävä'-funktio
        "LED Controller", // Nimi
        4096,                // Muistin varaus
        NULL,                // Parametrit
        1,                   // Suoritusprioriteetti
        NULL,                // Käsittelijä
        0                    // Ydin, jolle tehtävä annetaan; ESP:illä on 2
ydintä(!)
    );

}

void loop() {

    if (voltage < 0.01) {    //LED:in käyttöliittymä; jos jännite on nolla,
niin palaa punainen ledi
        digitalWrite(RED_PIN, HIGH);
        digitalWrite(GREEN_PIN, LOW);
    } else {
        digitalWrite(RED_PIN, LOW); //Nollasta poikkeavalla arvolla
vihreä
        digitalWrite(GREEN_PIN, HIGH);
    }
}

```

```

int currentCLK = digitalRead(CLK_PIN); //Lukee pulssianturin nykytilan

if (inMenu) { //Tarkistaa ollaanko alkuvalikossa

    if (currentCLK != lastCLK) { //Tarkistaa, onko pulssianturin tila
muuttunut

        if (digitalRead(DT_PIN) != currentCLK) { //Tarkistaa anturin
pyörimissuunnan
            menuIndex++; //Valikon osoitin myötäpäivään
        }
        else {
            menuIndex--; //Valikon osoitin vastapäivään
        }

        if (menuIndex > 3) menuIndex = 0; //Jos valikon osoitin pyrkii
siirtymään ruudun ulkopuolelle, osoitin palautetaan 1. riville
        if (menuIndex < 0) menuIndex = 3;

        drawMenu(); //Piirtää alkuvalikon uudella osoittimen paikalla
    }
}

lastCLK = currentCLK; //Siirtää piirtämiseen käytetyn anturin arvon
seuraavan loop()-kierroksen vertailuarvoksi

if (digitalRead(SW_PIN) == LOW) { //Lukee pulssianturin painikkeen tilan

    if (millis() - lastPress > buttonDebounce) { //Varmistaa, että
edellisestä painalluksesta on kulunut riittävästi aikaa

        lastPress = millis(); //Päivittää juuri tehdyn painalluksen
ajankohdan muistiin

        if (inMenu) {

            selectedOption = menuIndex; //Päivittää painikkeella
valituksi vaihtoehdoksi sen, jossa osoitin painallushetkellä on
            inMenu = false; //Poistutaan alkuvalikosta
            drawOption(); //Piirretään valittu mittausdata
        }
        else {

            inMenu = true; //Jos ei olla alkuvalikossa, painallus
palauttaa käyttäjän alkuvalikkoon
            drawMenu(); //Piirretään alkuvalikko
        }
    }
}

if (!inMenu) { //Ajetaan, jos LCD-ruutu esittää valitun vaihtoehdon
mittausdataa

    if (millis() - lastUpdate > updateInterval) { //Tarkistaa, onko
ruudun edellisestä päivityksestä kulunut riittävästi aikaa

```

```

        lastUpdate = millis(); //Jos päivitys tapahtuu, "nollataan"
laskuri

        drawOption(); //Piirtää mittausdatan uudelleen
    }
}
}

```

6. API:n linkit

Domain : albert2026.azurewebsites.net (sammutetaan 10.06.2026)

- Kaikkien tuuliturbiinien tiedot - domain/api/wind_turbines
- Yksittäisen tuuliturbiinin tiedot - [domain/api/wind_turbines/\(tuuliturbiinin ID\)](http://domain/api/wind_turbines/(tuuliturbiinin ID))
- Yksittäisen tuuliturbiinin kaikkien anturien tiedot
- [domain/api/wind_turbines/\(tuuliturbiinin ID\)/sensors](http://domain/api/wind_turbines/(tuuliturbiinin ID)/sensors)
- Yksittäisen tuuliturbiinin yksittäisen anturin tiedot
- [domain/api/wind_turbines/\(tuuliturbiinin ID\)/sensors/\(anturin ID\)](http://domain/api/wind_turbines/(tuuliturbiinin ID)/sensors/(anturin ID))
- Yksittäisen tuuliturbiinin kaikkien anturien mittaustiedot
- [domain/api/wind_turbines/\(tuuliturbiinin ID\)/sensors/logs/all](http://domain/api/wind_turbines/(tuuliturbiinin ID)/sensors/logs/all)
- Yksittäisen tuuliturbiinin yksittäisen anturien viimeiset mittaustiedot
- [domain/api/wind_turbines/\(tuuliturbiinin ID\)/sensors/\(anturin ID\)/logs](http://domain/api/wind_turbines/(tuuliturbiinin ID)/sensors/(anturin ID)/logs)

7. Kehitysideat

Generaattorin johtimet: Käytetään tavallisten johtimien ja mikrokytkimien sijaan liukurengasta (slip ringiä). Tällöin voimalan naselli voi vapaasti pyöriä vaakatasossa useita kierroksia ilman, että generaattorin johdot kiertyvät tai rajoittavat liikettä.

Jarrut: Estetään lukittuvalla järjestelmällä nasellia kääntymästä ulkoisten voimien vaikutuksesta (esim. kova tuuli). Tällöin järjestelmän käsitys turbiinin suunnasta pysyy synkronisoituna todellisen suunnan kanssa (sähkövikoja lukuun ottamatta), eikä erillistä anturia turbiinin asennon määrittämiseen välttämättä tarvita.

Tuulenmittarin sijainti: Sijoitetaan ideaalitapauksessa tuulen mittaus tuulivoimalan nasellin yläpuolelle, jolloin roottorin lapoihin osuvan tuulen mitattu suunta vastaa mahdollisimman tarkasti tuulen todellista suuntaa.

Latausjärjestelmä: Tuulivoimala käyttää kuutta 1.2 V ladattavaa paristoa nasellin kääntämiseen. Ilman latausjärjestelmää paristojen kemiallinen energia ehtyy nopeasti. Kehittyneemmässä projektiversiossa generaattorilla joko ladattaisiin järjestelmän paristoja tai sitä käytettäisiin suoraan stepperin osittaisena voimanlähteenä.

8. Lähteet

https://www.mouser.com/datasheet/2/588/AS5600_UG000254_2_00-1877354.pdf?srltid=AfmBOop892_kxxUgXnjFmKxcAdpxNOvunGEGbfWT3i0n2kUvdvzuPQ5A

<https://github.com/RobTillaart/AS5600>

<https://docs.arduino.cc/learn/electronics/stepper-motors/>

<https://www.kuisma.eu/elper/6puolijohteet/9puolijohde.html>